

CS787: GENERATIVE ARTIFICIAL INTELLIGENCE LARGE LANGUAGE MODELS

Arnab Bhattacharya
arnabb@cse.iitk.ac.in

Computer Science and Engineering,
Indian Institute of Technology, Kanpur
<http://web.cse.iitk.ac.in/~cs787/>

1st semester, 2026-27

Wed 12:00–13:15 at L20, Fri 12:00–13:15 at L19

Different Encoder and Decoder Architectures

- **Encoder** architectures give a *contextual embedding vector* for input text/token
 - Good for classification tasks
 - BERT

Different Encoder and Decoder Architectures

- **Encoder** architectures give a *contextual embedding vector* for input text/token
 - Good for classification tasks
 - BERT (Bi-Directional Encoder Representations from Transformers)

Different Encoder and Decoder Architectures

- **Encoder** architectures give a *contextual embedding vector* for input text/token
 - Good for classification tasks
 - BERT (Bi-Directional Encoder Representations from Transformers)
- **Decoder** architectures generate output text/token
 - Good for generative tasks
 - GPT

Different Encoder and Decoder Architectures

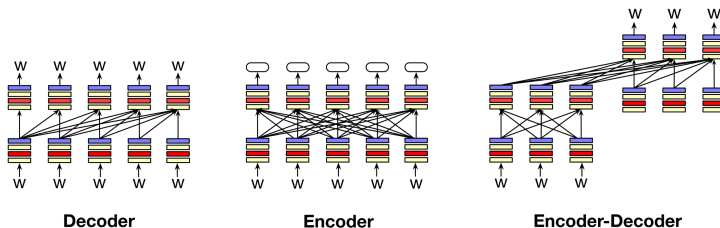
- **Encoder** architectures give a *contextual embedding vector* for input text/token
 - Good for classification tasks
 - BERT (Bi-Directional Encoder Representations from Transformers)
- **Decoder** architectures generate output text/token
 - Good for generative tasks
 - GPT (Generative Pre-Trained Transformer)

Different Encoder and Decoder Architectures

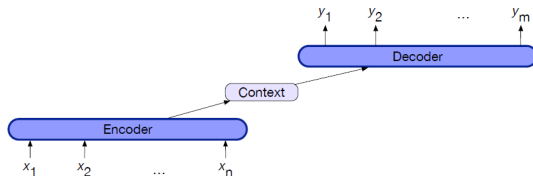
- **Encoder** architectures give a *contextual embedding vector* for input text/token
 - Good for classification tasks
 - BERT (Bi-Directional Encoder Representations from Transformers)
- **Decoder** architectures generate output text/token
 - Good for generative tasks
 - GPT (Generative Pre-Trained Transformer)
- **Encoder-Decoder** architectures produce embeddings for input and generate output
 - Good for translation tasks
 - T5

Different Encoder and Decoder Architectures

- **Encoder** architectures give a *contextual embedding vector* for input text/token
 - Good for classification tasks
 - BERT (Bi-Directional Encoder Representations from Transformers)
- **Decoder** architectures generate output text/token
 - Good for generative tasks
 - GPT (Generative Pre-Trained Transformer)
- **Encoder-Decoder** architectures produce embeddings for input and generate output
 - Good for translation tasks
 - T5 (Text-to-Text Transfer Transformer)

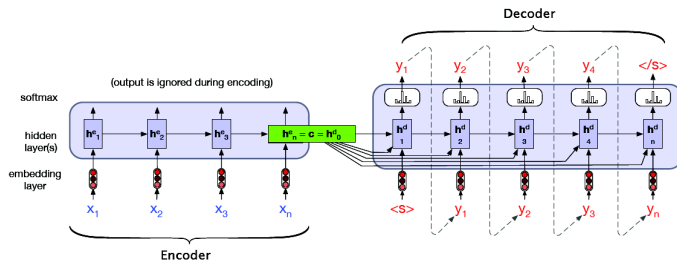


Encoder-Decoder Model



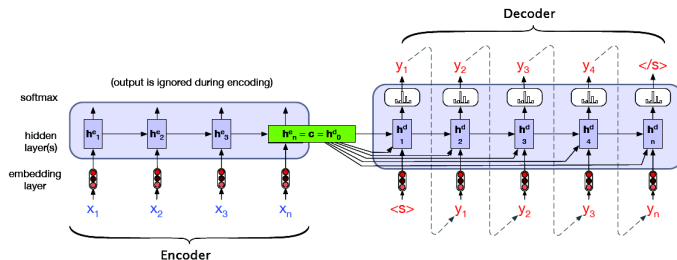
- **Encoder** accepts an input sequence and generates a sequence of contextualized representations
- **Context vector** is a function of the last contextual representation
 - This represents the entire sequence
- **Decoder** generates an arbitrary length sequence of output states

Encoder-Decoder using RNNs



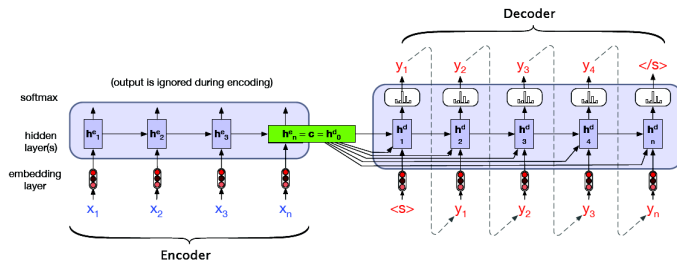
- Starts with a sentence beginning marker $<s>$
- Keeps generating till a sentence end marker $</s>$ is produced
- *Teacher forcing* is used

Encoder-Decoder using RNNs



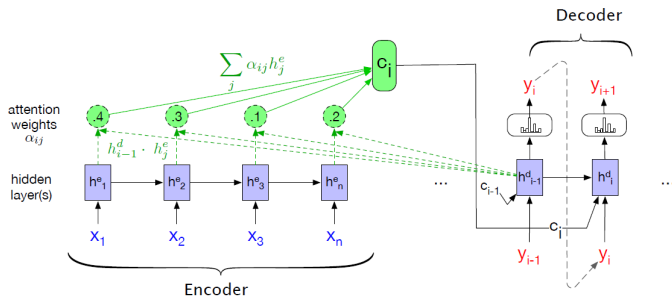
- Starts with a sentence beginning marker $\langle s \rangle$
- Keeps generating till a sentence end marker $\langle /s \rangle$ is produced
- *Teacher forcing* is used
- Only the last encoder state matters
 - **Information bottleneck**
 - Everything about the input sequence must be captured by it

Encoder-Decoder using RNNs



- Starts with a sentence beginning marker $\langle s \rangle$
- Keeps generating till a sentence end marker $\langle /s \rangle$ is produced
- *Teacher forcing* is used
- Only the last encoder state matters
 - **Information bottleneck**
 - Everything about the input sequence must be captured by it
- Can use *attention* mechanism to resolve

Encoder-Decoder with Attention

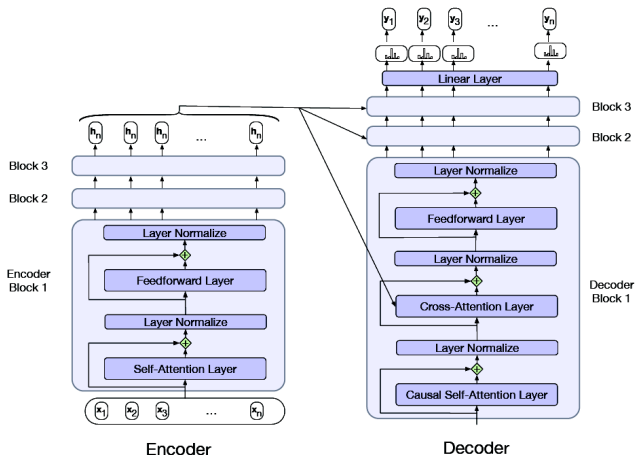


- Each decoder state gets an *attention* from every encoder state
- **Cross-attention**

$$\alpha_{ij} = \text{softmax}(h_{i-1}^d \cdot \vec{h}_j^e) \quad \forall j \in E$$

$$\vec{c}_i = \sum_{\forall j} (\alpha_{ij} \vec{h}_j^e)$$

Encoder-Decoder using Transformer Blocks



- Decoder blocks have both self-attention and cross-attention layers

Tokens versus Words

- What are words?

Tokens versus Words

- What are words?
- Tokens are processing units
- They should be words
 - Whitespace-separated
 - Inflectional and agglutinative languages

Tokens versus Words

- What are words?
- Tokens are processing units
- They should be words
 - Whitespace-separated
 - Inflectional and agglutinative languages
- *Statistical tokenization* algorithms generates *sub-word* level tokens

Tokens versus Words

- What are words?
- Tokens are processing units
- They should be words
 - Whitespace-separated
 - Inflectional and agglutinative languages
- *Statistical tokenization* algorithms generates *sub-word* level tokens
 - **Byte-Pair Encoding (BPE)** progressively merges two most *frequent* tokens

Tokens versus Words

- What are words?
- Tokens are processing units
- They should be words
 - Whitespace-separated
 - Inflectional and agglutinative languages
- *Statistical tokenization* algorithms generates *sub-word* level tokens
 - **Byte-Pair Encoding (BPE)** progressively merges two most *frequent* tokens
 - **WordPiece** progressively merges two most *likely* tokens

Tokens versus Words

- What are words?
- Tokens are processing units
- They should be words
 - Whitespace-separated
 - Inflectional and agglutinative languages
- *Statistical tokenization* algorithms generates *sub-word* level tokens
 - **Byte-Pair Encoding (BPE)** progressively merges two most *frequent* tokens
 - **WordPiece** progressively merges two most *likely* tokens
 - **SentencePiece (Unigram)** learns most likely split of character stream to tokens

Tokens versus Words

- What are words?
- Tokens are processing units
- They should be words
 - Whitespace-separated
 - Inflectional and agglutinative languages
- *Statistical tokenization* algorithms generates *sub-word* level tokens
 - **Byte-Pair Encoding (BPE)** progressively merges two most *frequent* tokens
 - **WordPiece** progressively merges two most *likely* tokens
 - **SentencePiece (Unigram)** learns most likely split of character stream to tokens
- Controlled vocabulary size
- Can be very different from actual words

- Byte-Pair Encoding (BPE) algorithm for *tokenization*

- **Byte-Pair Encoding (BPE)** algorithm for *tokenization*
- Start with every character as a token
- Merge two adjacent tokens with the *highest frequency* to form a new token

$$\arg \max \# \{t_1 t_2\}$$

- Sequences are first separated using whitespaces
 - Merge is not done across whitespaces
- Insert these tokens to the list and remove their constituents

- **Byte-Pair Encoding (BPE)** algorithm for *tokenization*
- Start with every character as a token
- Merge two adjacent tokens with the *highest frequency* to form a new token

$$\arg \max \# \{t_1 t_2\}$$

- Sequences are first separated using whitespaces
 - Merge is not done across whitespaces
- Insert these tokens to the list and remove their constituents
- Keep on repeating till a pre-fixed k number of tokens is reached

- **Byte-Pair Encoding (BPE)** algorithm for *tokenization*
- Start with every character as a token
- Merge two adjacent tokens with the *highest frequency* to form a new token

$$\arg \max \# \{t_1 t_2\}$$

- Sequences are first separated using whitespaces
 - Merge is not done across whitespaces
- Insert these tokens to the list and remove their constituents
- Keep on repeating till a pre-fixed k number of tokens is reached
- Unicode uses bytes as characters

- **Byte-Pair Encoding (BPE)** algorithm for *tokenization*
- Start with every character as a token
- Merge two adjacent tokens with the *highest frequency* to form a new token

$$\arg \max \# \{t_1 t_2\}$$

- Sequences are first separated using whitespaces
 - Merge is not done across whitespaces
- Insert these tokens to the list and remove their constituents
- Keep on repeating till a pre-fixed k number of tokens is reached
- Unicode uses bytes as characters
- Used by GPT2, LLaMa, GPT4, RoBERTa

BPE Example

- Initially

corpus

```
5   l o w _  
2   l o w e s t _  
6   n e w e r _  
3   w i d e r _  
2   n e w _
```

vocabulary

```
_, d, e, i, l, n, o, r, s, t, w
```

BPE Example

- Initially

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

- 'er' has the highest frequency

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

BPE Example

- Initially

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w

- 'er' has the highest frequency

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

_, d, e, i, l, n, o, r, s, t, w, er

- Next highest frequency is with 'er_', i.e., with word ending

corpus

5 l o w _
2 l o w e s t _
6 n e w e r _
3 w i d e r _
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er

BPE Example: More Steps

- Next is 'ne'

corpus

5 l o w _
2 l o w e s t _
6 n e w e r_
3 w i d e r_
2 n e w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er, ne

BPE Example: More Steps

- Next is 'ne'

corpus

5 l o w _
2 l o w e s t _
6 ne w er_
3 w i d er_
2 ne w _

vocabulary

, d, e, i, l, n, o, r, s, t, w, er, er, ne

- Finally

Merge

Current Vocabulary

(ne, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new
(l, o)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo
(lo, w)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low
(new, er_)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_
(low, _)	_, d, e, i, l, n, o, r, s, t, w, er, er_, ne, new, lo, low, newer_, low_

- WordPiece algorithm for *tokenization*

- **WordPiece** algorithm for *tokenization*
- Same as BPE with one difference
- Merge two adjacent tokens with the *highest likelihood* to form a new token

$$\arg \max (\#\{t_1 t_2\} / \#t_1 \cdot \#t_2)$$

- Sequences are first separated using whitespaces
 - Merge is not done across whitespaces

- **WordPiece** algorithm for *tokenization*
- Same as BPE with one difference
- Merge two adjacent tokens with the *highest likelihood* to form a new token

$$\arg \max (\#\{t_1 t_2\} / \#t_1 \cdot \#t_2)$$

- Sequences are first separated using whitespaces
 - Merge is not done across whitespaces
- Unicode uses bytes as characters

- **WordPiece** algorithm for *tokenization*
- Same as BPE with one difference
- Merge two adjacent tokens with the *highest likelihood* to form a new token

$$\arg \max (\#\{t_1 t_2\} / \#t_1 \cdot \#t_2)$$

- Sequences are first separated using whitespaces
 - Merge is not done across whitespaces
- Unicode uses bytes as characters
- Used by BERT, ELECTRA, DistilBERT

- SentencePiece framework for *tokenization*

SentencePiece

- **SentencePiece** framework for *tokenization*
- Does *not* separate words by whitespaces
- Considers *entire* sequence as one string
- Whitespace-separated words are designated by special character " _ "

SentencePiece

- **SentencePiece** framework for *tokenization*
- Does *not* separate words by whitespaces
- Considers *entire* sequence as one string
- Whitespace-separated words are designated by special character " _ "
- **SentencePiece BPE** then just proceeds in the same manner as BPE

- **SentencePiece** framework for *tokenization*
- Does *not* separate words by whitespaces
- Considers *entire* sequence as one string
- Whitespace-separated words are designated by special character " _ "
- **SentencePiece BPE** then just proceeds in the same manner as BPE
- **SentencePiece Unigram**
- Proceeds by deleting tokens that contribute the least to unigram probability likelihood
 - Can be approximated as lowest frequency

$$\arg \max t \in T \left(\prod P(t) \right)$$

- **SentencePiece** framework for *tokenization*
- Does *not* separate words by whitespaces
- Considers *entire* sequence as one string
- Whitespace-separated words are designated by special character " _ "
- **SentencePiece BPE** then just proceeds in the same manner as BPE
- **SentencePiece Unigram**
- Proceeds by deleting tokens that contribute the least to unigram probability likelihood
 - Can be approximated as lowest frequency

$$\arg \max t \in T \left(\prod P(t) \right)$$

- Keep on repeating till a pre-fixed k number of tokens is retained

- **SentencePiece** framework for *tokenization*
- Does *not* separate words by whitespaces
- Considers *entire* sequence as one string
- Whitespace-separated words are designated by special character " _ "
- **SentencePiece BPE** then just proceeds in the same manner as BPE
- **SentencePiece Unigram**
- Proceeds by deleting tokens that contribute the least to unigram probability likelihood
 - Can be approximated as lowest frequency

$$\arg \max t \in T \left(\prod P(t) \right)$$

- Keep on repeating till a pre-fixed k number of tokens is retained
- Better for languages that do not have word separators

- **SentencePiece** framework for *tokenization*
- Does *not* separate words by whitespaces
- Considers *entire* sequence as one string
- Whitespace-separated words are designated by special character " _ "
- **SentencePiece BPE** then just proceeds in the same manner as BPE
- **SentencePiece Unigram**
- Proceeds by deleting tokens that contribute the least to unigram probability likelihood
 - Can be approximated as lowest frequency

$$\arg \max t \in T \left(\prod P(t) \right)$$

- Keep on repeating till a pre-fixed k number of tokens is retained
- Better for languages that do not have word separators
- Used by T5, ALBERT, mBART, LLaMa

Measure of Tokenization

- How good is a tokenizer?

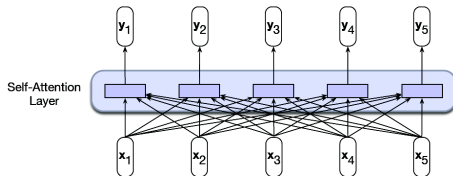
Measure of Tokenization

- How good is a tokenizer?
- **Fertility** measures the number of tokens (sub-words) produced per word

$$\frac{\sum_w t \in w}{|w|}$$

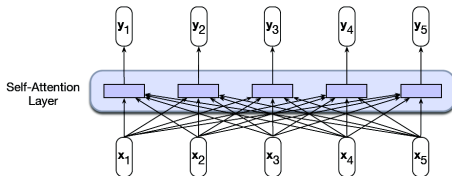
- Ideal fertility is 1
- More the fertility, more fragmented is the word, and worse is the tokenization

Bi-Directional Transformer Encoder



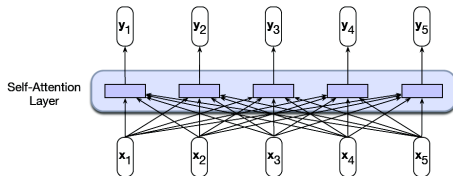
- **BERT** stands for *Bidirectional Encoder Representations from Transformer*
- Uses self-attention from both past and future

Bi-Directional Transformer Encoder



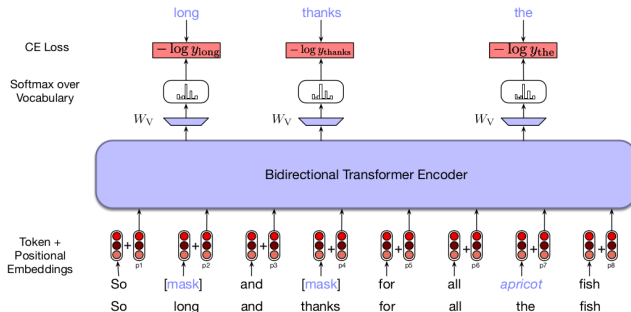
- **BERT** stands for *Bidirectional Encoder Representations from Transformer*
- Uses self-attention from both past and future
- Simple stacking of layers or using transformer blocks allows a time step to indirectly see itself

Bi-Directional Transformer Encoder



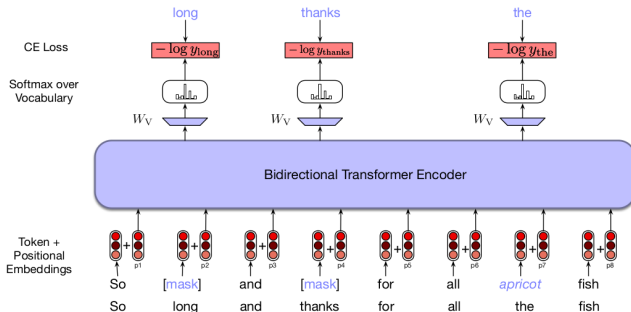
- **BERT** stands for *Bidirectional Encoder Representations from Transformer*
- Uses self-attention from both past and future
- Simple stacking of layers or using transformer blocks allows a time step to indirectly see itself
- Masking to resolve that

Masked Language Model (MLM)



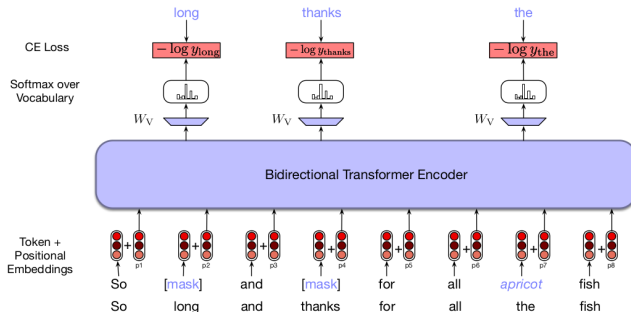
- Words are *masked*, i.e., they are replaced by special [MASK] tokens
- Sometimes they are replaced deliberately by *unrelated* words

Masked Language Model (MLM)



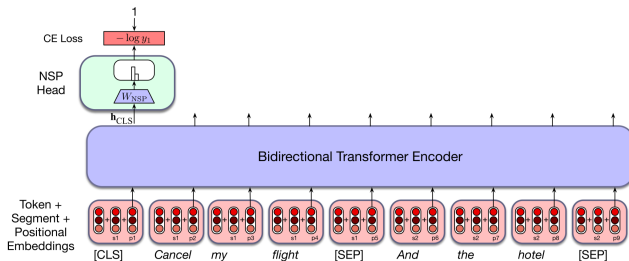
- Words are *masked*, i.e., they are replaced by special [MASK] tokens
- Sometimes they are replaced deliberately by *unrelated* words
- BERT uses **subword tokens** instead of actual words

Masked Language Model (MLM)



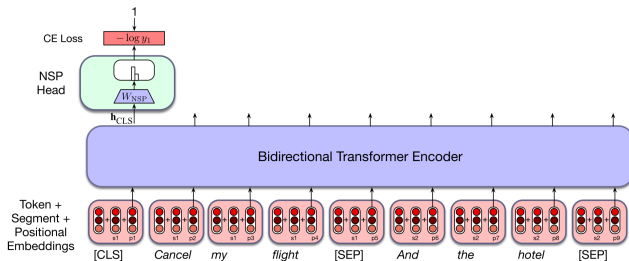
- Words are *masked*, i.e., they are replaced by special [MASK] tokens
- Sometimes they are replaced deliberately by *unrelated* words
- BERT uses **subword tokens** instead of actual words
- Produces **contextual embeddings**, i.e., embeddings of a word in context of the sequence

Next Sentence Prediction (NSP)



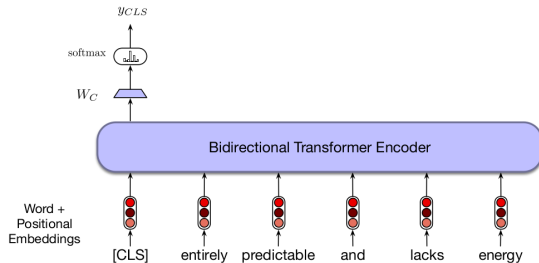
- **Next Sentence Prediction:** Given two sentences (or segments), predict if they are adjacent (related)
- Binary classification problem
- Special tokens to indicate separation of segments (actually end of segments)

Next Sentence Prediction (NSP)



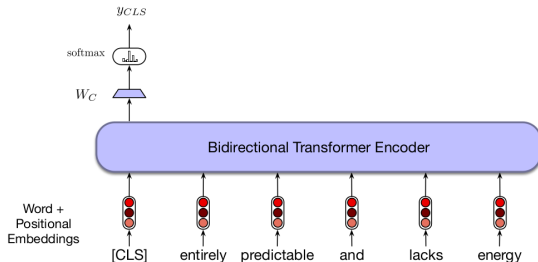
- **Next Sentence Prediction:** Given two sentences (or segments), predict if they are adjacent (related)
- Binary classification problem
- Special tokens to indicate separation of segments (actually end of segments)
- This objective improves embedding of sentences

Fine-tuning using BERT



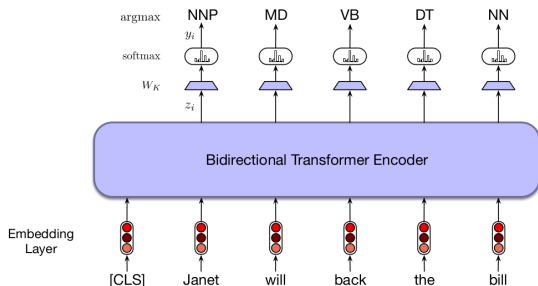
- For word tasks, the contextual embedding vector is used

Fine-tuning using BERT



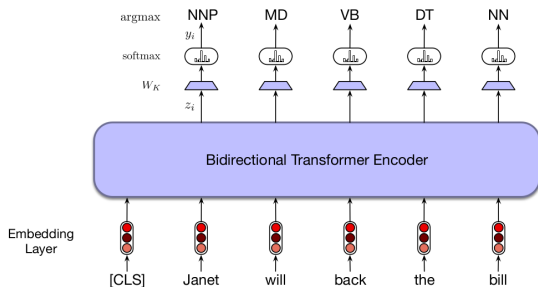
- For word tasks, the contextual embedding vector is used
- For sentence tasks, a special [CLS] token is prepended
- Vector of [CLS] is used for tasks with *task-specific training data*
 - Sentiment classification
 - Sentence entailment

Word Tasks using BERT



- Each individual *pre-trained* word vector is fine-tuned using a separate network for specific tasks
 - POS tagging
 - NER identification

Word Tasks using BERT



- Each individual *pre-trained* word vector is fine-tuned using a separate network for specific tasks
 - POS tagging
 - NER identification
- Since subwords may not be exactly aligned with words, a word is assigned the class to which its first subword belongs to
- Training assigns the golden class to all subwords

Temperature Sampling

- Softmax uses $\exp(x)$ and then normalizes it
- **Temperature** changes the distribution
- With temperature τ , softmax becomes $\exp(x/\tau)$ and then normalization
- With $\tau < 1$, distribution becomes more extreme
- With $\tau > 1$, distribution becomes more uniform

Temperature Sampling

- Softmax uses $\exp(x)$ and then normalizes it
- **Temperature** changes the distribution
- With temperature τ , softmax becomes $\exp(x/\tau)$ and then normalization
- With $\tau < 1$, distribution becomes more extreme
- With $\tau > 1$, distribution becomes more uniform
- Example
 - Logits $[1.2, 0.9, 0.1, -0.5]$
 - Normal softmax (i.e., $\tau = 1$) produces $[0.44, 0.33, 0.15, 0.08]$

Temperature Sampling

- Softmax uses $\exp(x)$ and then normalizes it
- **Temperature** changes the distribution
- With temperature τ , softmax becomes $\exp(x/\tau)$ and then normalization
- With $\tau < 1$, distribution becomes more extreme
- With $\tau > 1$, distribution becomes more uniform
- Example
 - Logits $[1.2, 0.9, 0.1, -0.5]$
 - Normal softmax (i.e., $\tau = 1$) produces $[0.44, 0.33, 0.15, 0.08]$
 - $\tau = 0.5$ produces $[0.59, 0.32, 0.07, 0.02]$

Temperature Sampling

- Softmax uses $\exp(x)$ and then normalizes it
- **Temperature** changes the distribution
- With temperature τ , softmax becomes $\exp(x/\tau)$ and then normalization
- With $\tau < 1$, distribution becomes more extreme
- With $\tau > 1$, distribution becomes more uniform
- Example
 - Logits $[1.2, 0.9, 0.1, -0.5]$
 - Normal softmax (i.e., $\tau = 1$) produces $[0.44, 0.33, 0.15, 0.08]$
 - $\tau = 0.5$ produces $[0.59, 0.32, 0.07, 0.02]$
 - $\tau = 2.0$ produces $[0.35, 0.30, 0.20, 0.15]$

Temperature Sampling

- Softmax uses $\exp(x)$ and then normalizes it
- **Temperature** changes the distribution
- With temperature τ , softmax becomes $\exp(x/\tau)$ and then normalization
- With $\tau < 1$, distribution becomes more extreme
- With $\tau > 1$, distribution becomes more uniform
- Example
 - Logits $[1.2, 0.9, 0.1, -0.5]$
 - Normal softmax (i.e., $\tau = 1$) produces $[0.44, 0.33, 0.15, 0.08]$
 - $\tau = 0.5$ produces $[0.59, 0.32, 0.07, 0.02]$
 - $\tau = 2.0$ produces $[0.35, 0.30, 0.20, 0.15]$
- With random top- k sampling, temperature can have a large effect

Large Language Model (LLM)

- Pre-training (PT)

- Using the *language modeling* task of predicting the next token
- Can be automatically generated from a corpus

Large Language Model (LLM)

- **Pre-training (PT)**
 - Using the *language modeling* task of predicting the next token
 - Can be automatically generated from a corpus
- **Continual pre-training (CPT)**
 - Continue pre-training, but with private or additional data

Large Language Model (LLM)

- **Pre-training (PT)**

- Using the *language modeling* task of predicting the next token
- Can be automatically generated from a corpus

- **Continual pre-training (CPT)**

- Continue pre-training, but with private or additional data

- **Supervised Fine-Tuning (SFT)**

- Needs a *task-specific* training data
- A set of **prompts** and their corresponding **responses**
 - Prompt: Tell me about Sukumar Ray
 - Response: Sukumar Ray was a poet ...
⟨written by a human expert⟩

Large Language Model (LLM)

- **Pre-training (PT)**
 - Using the *language modeling* task of predicting the next token
 - Can be automatically generated from a corpus
- **Continual pre-training (CPT)**
 - Continue pre-training, but with private or additional data
- **Supervised Fine-Tuning (SFT)**
 - Needs a *task-specific* training data
 - A set of **prompts** and their corresponding **responses**
 - Prompt: Tell me about Sukumar Ray
 - Response: Sukumar Ray was a poet ...*<written by a human expert>*
 - Uses the same next token prediction for *generating* the response

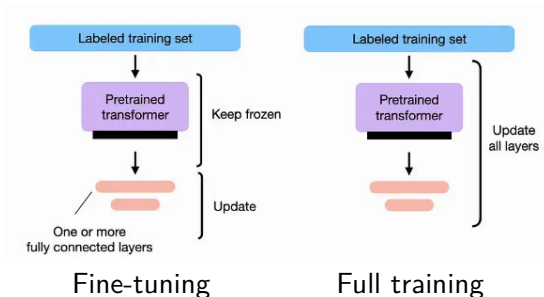
Large Language Model (LLM)

- **Pre-training (PT)**
 - Using the *language modeling* task of predicting the next token
 - Can be automatically generated from a corpus
- **Continual pre-training (CPT)**
 - Continue pre-training, but with private or additional data
- **Supervised Fine-Tuning (SFT)**
 - Needs a *task-specific* training data
 - A set of **prompts** and their corresponding **responses**
 - Prompt: Tell me about Sukumar Ray
 - Response: Sukumar Ray was a poet ...*<written by a human expert>*
 - Uses the same next token prediction for *generating* the response
- **Instruction Fine-Tuning (IFT)**
 - **Reward Model Training (RMT)**
 - **Reinforcement Learning from Human Feedback (RLHF)**

Large Language Model (LLM)

- **Pre-training (PT)**
 - Using the *language modeling* task of predicting the next token
 - Can be automatically generated from a corpus
- **Continual pre-training (CPT)**
 - Continue pre-training, but with private or additional data
- **Supervised Fine-Tuning (SFT)**
 - Needs a *task-specific* training data
 - A set of **prompts** and their corresponding **responses**
 - Prompt: Tell me about Sukumar Ray
 - Response: Sukumar Ray was a poet ...*<written by a human expert>*
 - Uses the same next token prediction for *generating* the response
- **Instruction Fine-Tuning (IFT)**
 - **Reward Model Training (RMT)**
 - **Reinforcement Learning from Human Feedback (RLHF)**
- **Retrieval Augmented Generation (RAG)**
 - Retrieve information from an external knowledge source

Supervised Fine-Tuning



- **Supervised fine-tuning** (or simply **fine-tuning**) allows contextual *pre-trained* word vectors to be used for downstream CL/NLP/IR tasks
- Task-specific training data is needed
- Updates only the last layers

Continued Pre-Training

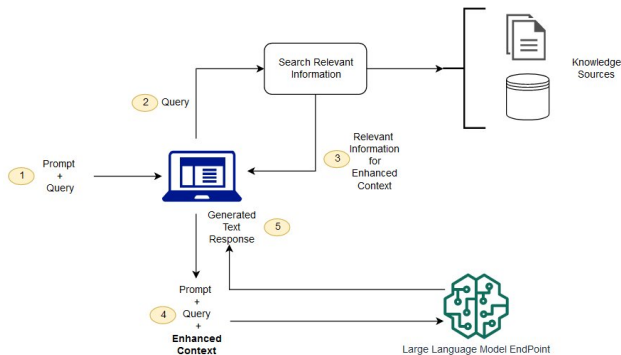


vs



- **Continued pre-training (CPT)** allows updating weights based on *new* training data
- Domain adaptation
- Updates all the layers

Retrieval Augmented Generation (RAG)



- **Retrieval augmented generation (RAG)** improves answer quality by augmenting the context
- Typically, an (external) knowledge source is indicated, from which *information retrieval (IR)* methods retrieve relevant passages

Instruction Fine-Tuning

- Early GPT systems
 - Prompt: “Translate to French: The small dog”

Instruction Fine-Tuning

- Early GPT systems
 - Prompt: “Translate to French: The small dog”
 - Response: “The small dog crossed the road.”
- Auto-regression becomes more important than *instruction*

Instruction Fine-Tuning

- Early GPT systems
 - Prompt: “Translate to French: The small dog”
 - Response: “The small dog crossed the road.”
- Auto-regression becomes more important than *instruction*
- Can also generate *dangerous, fake, toxic, stereotypic* and *biased* responses

Instruction Fine-Tuning

- Early GPT systems
 - Prompt: “Translate to French: The small dog”
 - Response: “The small dog crossed the road.”
- Auto-regression becomes more important than *instruction*
- Can also generate *dangerous, fake, toxic, stereotypic* and *biased* responses
- *Model alignment* aligns to human needs
 - Goal is more to see how well instructions are followed

Instruction Fine-Tuning

- Early GPT systems
 - Prompt: “Translate to French: The small dog”
 - Response: “The small dog crossed the road.”
- Auto-regression becomes more important than *instruction*
- Can also generate *dangerous, fake, toxic, stereotypic* and *biased* responses
- *Model alignment* aligns to human needs
 - Goal is more to see how well instructions are followed
- *Preference alignment* aligns to human preferences

RMT (Reward Model Training)

- Uses the pre-trained model as initial base model
- Given a prompt and a response, the aim is to output a *score*

RMT (Reward Model Training)

- Uses the pre-trained model as initial base model
- Given a prompt and a response, the aim is to output a *score*
- Given a prompt, *multiple* responses are generated

RMT (Reward Model Training)

- Uses the pre-trained model as initial base model
- Given a prompt and a response, the aim is to output a *score*
- Given a prompt, *multiple* responses are generated
- Human expert (or its proxy) *ranks* these responses
 - Does *not* generate absolute scores
 - May use majority ranking if there are multiple human experts

RMT (Reward Model Training)

- Uses the pre-trained model as initial base model
- Given a prompt and a response, the aim is to output a *score*
- Given a prompt, *multiple* responses are generated
- Human expert (or its proxy) *ranks* these responses
 - Does *not* generate absolute scores
 - May use majority ranking if there are multiple human experts
- RM uses *pairwise rankings*: $\binom{n}{2}$ for n responses
- Generates *embeddings* of responses

Embeddings of Responses

- Generates *embeddings* of responses
- Probability of choosing preferred response r_{pref} (with embedding z_{pref}) with respect to rejected response r_{rej} (z_{rej}) should be *maximized*
- Using log-odds

$$\delta = \log \frac{P(r_{\text{pref}} \succ r_{\text{rej}})}{P(r_{\text{rej}} \succ r_{\text{pref}})}$$

$$P(r_{\text{pref}} \succ r_{\text{pref}}) = \frac{\exp(\delta)}{1 + \exp(\delta)} = \sigma(z_{\text{pref}} - z_{\text{rej}})$$

Embeddings of Responses

- Generates *embeddings* of responses
- Probability of choosing preferred response r_{pref} (with embedding z_{pref}) with respect to rejected response r_{rej} (z_{rej}) should be *maximized*
- Using log-odds

$$\delta = \log \frac{P(r_{\text{pref}} \succ r_{\text{rej}})}{P(r_{\text{rej}} \succ r_{\text{pref}})}$$

$$P(r_{\text{pref}} \succ r_{\text{pref}}) = \frac{\exp(\delta)}{1 + \exp(\delta)} = \sigma(z_{\text{pref}} - z_{\text{rej}})$$

- Reward function RM that assigns embedding as score to response r for prompt P

$$P(r_{\text{pref}} \succ r_{\text{rej}}) = \sigma(RM(P, r_{\text{pref}}), RM(P, r_{\text{rej}}))$$

Embeddings of Responses

- Generates *embeddings* of responses
- Probability of choosing preferred response r_{pref} (with embedding z_{pref}) with respect to rejected response r_{rej} (z_{rej}) should be *maximized*
- Using log-odds

$$\delta = \log \frac{P(r_{\text{pref}} \succ r_{\text{rej}})}{P(r_{\text{rej}} \succ r_{\text{rej}})}$$

$$P(r_{\text{pref}} \succ r_{\text{pref}}) = \frac{\exp(\delta)}{1 + \exp(\delta)} = \sigma(z_{\text{pref}} - z_{\text{rej}})$$

- Reward function RM that assigns embedding as score to response r for prompt P

$$P(r_{\text{pref}} \succ r_{\text{rej}}) = \sigma(RM(P, r_{\text{pref}}), RM(P, r_{\text{rej}}))$$

- Loss is negative log-likelihood

$$L = -\log \sigma(RM(P, r_{\text{pref}}), RM(P, r_{\text{rej}}))$$

- This trains the reward model RM

RLHF (Reinforcement Learning from Human Feedback)

- Human feedback is simulated by using the reward model
- Essentially, the SFT model is further fine-tuned using the RM model
- For a prompt, SFT model is encouraged to generate a response with a high score
- Find the *best* model Π_{RL} generated by RL from SFT

$$\max_{\Pi_{RL}} RM(\Pi_{RL}(Response|Prompt))$$

RLHF (Reinforcement Learning from Human Feedback)

- Human feedback is simulated by using the reward model
- Essentially, the SFT model is further fine-tuned using the RM model
- For a prompt, SFT model is encouraged to generate a response with a high score
- Find the *best* model Π_{RL} generated by RL from SFT

$$\max_{\Pi_{RL}} RM(\Pi_{RL}(Response|Prompt))$$

- However, RM may start forgetting original LM and give high scores for poor responses

RLHF (Reinforcement Learning from Human Feedback)

- Human feedback is simulated by using the reward model
- Essentially, the SFT model is further fine-tuned using the RM model
- For a prompt, SFT model is encouraged to generate a response with a high score
- Find the *best* model Π_{RL} generated by RL from SFT

$$\max_{\Pi_{RL}} RM(\Pi_{RL}(\text{Response}|\text{Prompt}))$$

- However, RM may start forgetting original LM and give high scores for poor responses
- Response should not deviate too much from the original SFT model
- *Penalty* term to capture deviation
- If new model is Π_{RL} and original model is Π_{SFT}

$$\max_{\Pi_{RL}} [RM(\Pi_{RL}(\text{Response}|\text{Prompt})) - \beta \cdot KL(\Pi_{RL}, \Pi_{SFT})]$$

How Large is a Large Language Model?

- Performance of LLMs depend mostly on
 - Model size
 - Dataset size
 - Amount of compute
- **Scaling laws** to determine relationships

How Large is a Large Language Model?

- Performance of LLMs depend mostly on
 - Model size
 - Dataset size
 - Amount of compute
- **Scaling laws** to determine relationships
- Number of parameters depend roughly as
 - Quadratic on vector dimensions
 - Linear on number of transformer blocks
- With $d = 10k$, $n = 10$, total number of parameters is
$$N = 10 \times 10^4 \times 10^4 \times 10 = 10^{10} = 10 \text{ billion}$$

Perplexity

- **Perplexity** is an *intrinsic* measure of how good an LLM is
- *Extrinsic* measures evaluate LLMs on downstream tasks

Perplexity

- **Perplexity** is an *intrinsic* measure of how good an LLM is
- *Extrinsic* measures evaluate LLMs on downstream tasks
- If the predicted next word has higher probability, then it is better

$$P(w_{1:n}) = P(w_1)P(w_2|w_1)P(w_3|w_{1:2}) \dots P(w_n|w_{1:n-1}) = \prod_{i=1}^n P(w_i|w_{<i})$$

- *Perplexity* normalizes by sequence length
- For a model with parameters θ for a test sequence w of N tokens

$$\text{Perplexity}_\theta = P_\theta(w_{1:N})^{-\frac{1}{N}}$$

Perplexity

- **Perplexity** is an *intrinsic* measure of how good an LLM is
- *Extrinsic* measures evaluate LLMs on downstream tasks
- If the predicted next word has higher probability, then it is better

$$P(w_{1:n}) = P(w_1)P(w_2|w_1)P(w_3|w_{1:2}) \dots P(w_n|w_{1:n-1}) = \prod_{i=1}^n P(w_i|w_{<i})$$

- *Perplexity* normalizes by sequence length
- For a model with parameters θ for a test sequence w of N tokens

$$\text{Perplexity}_\theta = P_\theta(w_{1:N})^{-\frac{1}{N}}$$

- Changed to log space for easier calculations

$$\text{Perplexity}_\theta = \exp \left\{ -\frac{1}{N} \sum_{i=1}^N \log p_\theta(w_i|w_{<i}) \right\}$$

- Lower the perplexity, better the model
- Tokenizer plays a big role

Efficiency in LLMs

- **KV Cache:** Key and value vectors remain same, and are cached

Efficiency in LLMs

- **KV Cache:** Key and value vectors remain same, and are cached
- **Parameter Efficient Finetuning (PEFT)**
 - Only certain parameters are trained

Efficiency in LLMs

- **KV Cache:** Key and value vectors remain same, and are cached
- **Parameter Efficient Finetuning (PEFT)**
 - Only certain parameters are trained
- **Low Rank Adaptation (LoRA)**
 - Weight matrices W^Q, W^K, W^V, W^O , etc. of size $N \times d$ get updated by ΔW of same size
 - Freezes pre-trained weights
 - Subsequent weight updates are $W + \Delta W$
 - Done through two smaller matrices of rank $r \ll N, d$ obtained through *low-rank decomposition*

$$\Delta W_{N \times d} \approx A_{N \times r} B_{r \times d}$$

- During fine-tuning and inferencing, only A, B matrices are used

Problems of LLM Chatbots

- **Hallucination**

- Wrong answers
- Mix-up of languages
- Indian languages not well supported

Problems of LLM Chatbots

- **Hallucination**

- Wrong answers
- Mix-up of languages
- Indian languages not well supported

- **Prompt engineering**

- Highly sensitive to prompts
- A slight change in prompt can generate a highly different response

Problems of LLM Chatbots

- **Hallucination**
 - Wrong answers
 - Mix-up of languages
 - Indian languages not well supported
- **Prompt engineering**
 - Highly sensitive to prompts
 - A slight change in prompt can generate a highly different response
- **Jail breaking**
 - Using AI to do immoral/illegal stuff
 - How to hack into a phone having Android version 9

Problems of LLM Chatbots

- **Hallucination**
 - Wrong answers
 - Mix-up of languages
 - Indian languages not well supported
- **Prompt engineering**
 - Highly sensitive to prompts
 - A slight change in prompt can generate a highly different response
- **Jail breaking**
 - Using AI to do immoral/illegal stuff
 - How to hack into a phone having Android version 9
- **Prompt injection**
 - Reveal more information that is intended to be revealed
 - Use them for other purposes